

```

\documentclass{article}
\usepackage{graphicx} % Required for inserting images
\usepackage{graphicx} % Required for inserting images
\usepackage{geometry}
\usepackage{amsmath}
\usepackage{tikz}
\usepackage{tikz-qtrees}
\usepackage{algorithm}
\usepackage{algorithmic}
\usepackage{wrapfig}
\usepackage{CJK}
\usepackage{indentfirst}
\usepackage{amsmath}
\usepackage{cases}
\usepackage{amsmath,amsfonts}
\usepackage{setspace}
\usepackage{tkz-graph}
\onehalfspacing
\title{A2}
\author{Changze Wu \texttt{\textasciitilde 1008837840}}
\date{March 2024}
\begin{document}
\maketitle

\section{Q1}
Proof:\par
\begin{wrapfigure}{r}{4cm}
\centering
\flushright \begin{tikzpicture}[semithick, ->, thick,scale=0.9, every
node/.style={scale=1}]
\Tree
[
.\node[draw, circle]{v};
[.\node[draw, circle]{x};
[.T ] [.S ]
] [.R ]
]
\end{tikzpicture}
\caption{\footnotesize Diagram}
\end{wrapfigure}
Let  $r = \text{size}(R)$ ,  $s = \text{size}(S)$ ,  $t = \text{size}(T)$  at the time of the rotation.  $v$  is
left-heavy, so either
\begin{itemize}
\item a node was added to  $x$  to cause imbalance, or
\item a node was removed from  $R$  to cause imbalance.
\end{itemize}
Assumption:
\begin{itemize}
\item  $\text{weight}(v.\text{left}) \geq \text{weight}(v.\text{right}) * 3$   $\Rightarrow$   $3(r+1) \leq s + t + 2$ 
\item  $3(r+1) \leq s + t + 2$ 
\end{itemize}
\textbf{Case 1:} Before addition, we had a WBT:
\begin{itemize}
\item  $r+1 \leq 3(s+t+1)$  and  $s+t+1 \leq 3(r+1)$ 
\item  $t \leq 3(s+1)$  and  $s \leq 3(t+1)$ 
\end{itemize}
From the above, we conclude below:
\begin{itemize}
\item  $3r+1 \leq s+t \leq 3r+2$ 
\item  $s \leq 2t+1$ 
\item  $r \leq 3s+3t+2 \leq 12t+11$ 
\item  $t \leq 3s+3$ 
\item  $s \leq 3t+3$ 
\end{itemize}

```

Since s, t, r are all integer, we get $s+t = 3r+2$. $\frac{1}{3}(t+1) \leq s + \frac{1}{3}(s+t-1) \leq s+2+r$, for $r \geq 0$. Since $3r+1 \leq s+t \Rightarrow r \leq \frac{s+t-1}{3} \Rightarrow \frac{4(2t+1)+t+5}{3} = 3t+3$, for $s \leq 2t+1$. Hold $\frac{1}{3} \leq \frac{t+1}{s+r+2} \leq 3$. Since $r \leq \frac{s+t-1}{3}$ and $t \leq 3s+3 \Rightarrow r+1 \leq \frac{s+3s+3-1}{3}+1 = \frac{4s+2}{3}+1 \leq 3s+3$, for $s \geq 0$. Since $s+1 \leq s+t+1 \leq 3r+3$, for $t \geq 0$. Hold $\frac{1}{3} \leq \frac{r+1}{s+1} \leq 3$.

Case 2: Before removal, we had a WBT:

- $r+2 \leq 3(s+t+2)$ and $s+t+2 \leq 3(r+2)$
- $s+1 \leq 3(t+1)$ and $t+1 \leq 3(s+1)$

From the above, we conclude below:

- $3r+1 \leq s+t \leq 3r+4$
- $s \leq 2t+1$
- $r \leq 3s+3t+4$
- $s \leq 3t+2$
- $t \leq 3s+2$

Since $\frac{1}{3}(t+1) \leq s+1 \leq s+2+r$, for $r \geq 0$, $t \leq 3s+2$. Since $3r+1 \leq s+t \Rightarrow r \leq \frac{s+t-1}{3} \Rightarrow \frac{4(2t+1)+t+5}{3} = 3t+3$, for $s \leq 2t+1$. Hold $\frac{1}{3} \leq \frac{t+1}{s+r+2} \leq 3$. Since $r \leq \frac{s+t-1}{3}$ and $t \leq 3s+2 \Rightarrow r+1 \leq \frac{s+3s+2-1}{3}+1 = \frac{4s+1}{3}+1 \leq 3s+3$, for $s \geq 0$. Since $3r+1 \leq s+t \leq 3t+1 \Rightarrow r \leq t-1$, and it is removal, for $r \geq 1$, $s+1 \leq s+t-1 \leq 3r+3$. Hold $\frac{1}{3} \leq \frac{r+1}{s+1} \leq 3$.

Both case hold, QED.

newpage

section{Q2}

subsection{}

Define two struct type of named `\textbf{node}` and `\textbf{func}`.

`\textbf{node}`: contains two int value named `\textit{key}` and `\textit{size}` (the function rebalance will use it but i haven't given it in my below code) and two node pointer value `\textit{left}` and `\textit{right}` to indicate left and right child.

`\textbf{func}`: contains two node pointer value named `\textit{left}` and `\textit{right}` to indicate `\textit{ST}_{<k}` and `\textit{ST}_{>k}`, and also have one boolean value `b` to indicate if the key in origin WBT.

And there are also to function named `\textbf{*newFunc}(node left, node right, bool b)` and `\textbf{*newNode}(int key)` each parameter corresponding in the definition.

There are also a function named `\textbf{balance}(node)` to re-balance the tree.

There

```

\begin{algorithm}
\caption{struct func split(node *T,int k)}
\begin{algorithmic}[1]
\STATE struct func func
\IF{ T == nil}
\STATE func\rightarrow left = NULL
\STATE func\rightarrow right = NULL
\STATE b = false
\STATE return result
\ENDIF
\IF{k < T\rightarrow key}
\STATE struct func leftsplit = split(T\rightarrow left, k)
\STATE func\rightarrow left = leftsplit\rightarrow left
\STATE func\rightarrow right = join(leftsplit\rightarrow right, T\rightarrow right)
\STATE func\rightarrow b = leftsplit\rightarrow b
\STATE balance(func\rightarrow left)
\STATE balance(func\rightarrow right)

```

```

\STATE return func
\ELSIF{k >= T\rightarrow$key}
\STATE struct func rightsplit = split(T\rightarrow$right, k)
\STATE func\rightarrow$left = join(T\rightarrow$left, rightsplit\rightarrow$left)
\STATE func\rightarrow$right = rightsplit\rightarrow$right
\STATE func\rightarrow$b = rightsplit\rightarrow$b
\STATE balance(func\rightarrow$left)
\STATE balance(func\rightarrow$right)
\STATE return func
\ELSE
\STATE func\rightarrow$left = T\rightarrow$left
\STATE func\rightarrow$right = T\rightarrow$right
\STATE func\rightarrow$b = true
\STATE balance(func\rightarrow$left)
\STATE balance(func\rightarrow$right)
\STATE return func;
\ENDIF
\end{algorithmic}
\end{algorithm}
\newpage
\subsection{}
Aside the \textbf{node} type we create before. Currently create a new \textbf{pair} which contained two \textbf{node} pointer variable named \textbf{tree} and \textbf{max}
\begin{algorithm}
\caption{struct Pair remove\_max(node* T)}
\begin{algorithmic}[1]
\STATE struct pair result;
\IF{T == NULL}
\STATE result\rightarrow$tree = NULL
\STATE result\rightarrow$max = NULL
\STATE return result;
\ENDIF
\IF{T\rightarrow$right == NULL}
\STATE result\rightarrow$tree = T\rightarrow$left;
\STATE result\rightarrow$max = T;
\STATE return result
\ENDIF
\STATE struct pair rightsub = remove\_max(T\rightarrow$right);
\STATE T\rightarrow$right = rightsub\rightarrow$tree;
\STATE result\rightarrow$tree = balance(T);
\STATE result\rightarrow$max = rightsub\rightarrow$max;
\STATE return result;
\end{algorithmic}
\end{algorithm}

\newpage
\subsection{}
\begin{algorithm}
\caption{struct node *join(struct node* L, struct node* G)}
\begin{algorithmic}[1]
\IF{L == NULL}
\STATE return G
\ENDIF
\IF{G == NULL}
\STATE return L
\ENDIF
\STATE struct pair result = remove\_Max(L)
\STATE struct Node* L\_prime = result\rightarrow$tree;
\STATE Node* max = result\rightarrow$max
\STATE struct Node* T = join(L\_prime, G)
\STATE T = insert(T, max)
\STATE return T

```

```

\end{algorithmic}
\end{algorithm}

\begin{algorithm}
\caption{struct Node* insert(struct node* root, struct node* new)}
\begin{algorithmic}[1]
\IF{root == NULL}
\STATE return new
\ENDIF
\IF{new == NULL}
\STATE return root
\ENDIF
\IF{new$\rightarrow$key < root$\rightarrow$key}
\STATE root$\rightarrow$left = insert(root$\rightarrow$left, new);
\ELSE
\STATE root$\rightarrow$right = insert(root$\rightarrow$right, new)
\ENDIF
\STATE root = balance(root)
\STATE return root
\end{algorithmic}
\end{algorithm}

\begin{algorithm}
\caption{void balance(node* T)}
\begin{algorithmic}[1]
\IF{weight(v.right) > weight(v.left) * 3:}
\STATE let x = v.right
\IF{weight(x.left) < weight(x.right) * 2:}
\STATE single rotation: counter-clockwise
\ELSE
\STATE double rotation: clockwise then counter-clockwise
\ENDIF
\ELSIF{weight(v.left) > weight(v.right) * 3:}
\STATE let x = v.left
\IF{weight(x.right) < weight(x.left) * 2:}
\STATE single rotation: clockwise
\ELSE
\STATE double rotation: counter-clockwise then clockwise
\ENDIF
\ELSE
\STATE no rotation
\ENDIF
\end{algorithmic}
\end{algorithm}

\begin{algorithm}
\caption{struct node* intersection(struct node *T1, struct node *T2)}
\begin{algorithmic}[1]
\STATE struct node *T = malloc(sizeof(struct node))
\STATE T = NULL
\IF{T1 == NULL || T2 == NULL}
\STATE return NULL;
\ENDIF
\STATE int array[1024]
\STATE createArray(T1, \&array, 0)
\STATE int i = 0;
\WHILE{isdigit(array[i])}
\STATE T = insert(T, search(T2, array[i]))
\STATE i++
\ENDWHILE
\STATE return T
\end{algorithmic}
\end{algorithm}

```

```

\begin{algorithm}
  \caption{void createArray(struct node *T, int *array, int index)}
  \begin{algorithmic}[1]
    \WHILE{T != NULL}
      \STATE array[index] = T.key
      \STATE createArray(T.left, \&array, index+1)
      \STATE createArray(T.right, \&array, index+1)
    \ENDWHILE
  \end{algorithmic}
\end{algorithm}

```

```

\begin{algorithm}
  \caption{node * search(struct node *T, int key)}
  \begin{algorithmic}[1]
    \IF{root == NULL}
      \STATE return NULL;
    \ENDIF
    \IF{T.key == key}
      \STATE return T;
    \ENDIF
    \IF{T.key > key}
      \STATE return search(T.left, key);
    \ELSE
      \STATE return search(T.right, key);
    \ENDIF
  \end{algorithmic}
\end{algorithm}

```

\clearpage

\section{Q4}

```

\begin{tabular}{c c c}
  \begin{minipage}{.22 \linewidth}
    Adjacency-list for left graph\\
    \begin{tabular}{l | l}
      1 & $\rightarrow$ 2 $\rightarrow$ 4 $\rightarrow$ 3\\
      2 & $\rightarrow$ 5 $\rightarrow$ 6\\
      4 & $\rightarrow$ 7\\
      3 & \\
      5 & \\
      6 & \\
      7 & \\
    \end{tabular}
  \end{minipage} &
  \begin{minipage}{.22 \linewidth}
    Adjacency-list for right graph \\
    \begin{tabular}{l | l}
      1 & $\rightarrow$ 4 $\rightarrow$ 3 $\rightarrow$ 2\\
      4 & $\rightarrow$ 6 $\rightarrow$ 7\\
      3 & $\rightarrow$ 5\\
      2 & \\
      6 & \\
      7 & \\
      5 & \\
    \end{tabular}
  \end{minipage} &
  \begin{minipage}{.22 \linewidth}
    Adjacency-list for wrong graph\\
    \begin{tabular}{l | l}
      1 & $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4\\
      2 & $\rightarrow$ 5 $\rightarrow$ 6\\
      3 & $\rightarrow$ 5 $\rightarrow$ 7\\
      4 & $\rightarrow$ 6 $\rightarrow$ 7\\
      5 & \\
    \end{tabular}
  \end{minipage}
\end{tabular}

```

```

6 &\\
7 &\\
\end{tabular}
\end{minipage}
\end{tabular}

```

According to third adjacency list, whenever first pass 2 or 3 or 4, the first should have two child, it is impossible for each one have one child.

```

\newpage
\section{Q5}
\subsection{}
\begin{table}[h!]
\centering
\begin{tabular}{|c|c|}
\hline
visit & enqueue\\
\hline\hline
a &  $\rightarrow$  b\\
b &  $\rightarrow$  a\\
c &  $\rightarrow$  g  $\rightarrow$  b\\
d &  $\rightarrow$  c\\
e &  $\rightarrow$  f  $\rightarrow$  a\\
f &  $\rightarrow$  c  $\rightarrow$  a\\
g &  $\rightarrow$  h  $\rightarrow$  f\\
h &  $\rightarrow$  d\\
\hline
\end{tabular}
\end{table}

```

```

\subsection{}
\begin{table}[h!]
\centering
\begin{tabular}{|c|c|c|c|c|c|c|c|c|}
\hline
& a & b & c & d & h & g & f & e\\
\hline\hline
discover & 1 & 2 & 3 & 4 & 5 & 6 & 10 & 11 \\
\hline\hline
finish & 16 & 15 & 14 & 9 & 8 & 7 & 13 & 12 \\
\hline
\end{tabular}
\caption{a  $\rightarrow$  b  $\rightarrow$  c  $\rightarrow$  f  $\rightarrow$  e  $\rightarrow$  d  $\rightarrow$  h  $\rightarrow$  g}
\end{table}

```

```

\subsection{}
\{a, b\} \{c, g, h, d\} \{e\} \{f\}

```

```

\subsection{}
\begin{tikzpicture}[scale=0.8]
\node[draw,circle] (a) at (0,0) {e};
\node[draw,circle] (b) at (2,0) {f};
\node[draw,circle] (c) at (2,2) {c, g, h, d};
\node[draw,circle] (d) at (0,2) {a, b};

\draw[<-] (a) -- (b) node [at start]{};
\draw[<-] (a) -- (d) node [at start]{};
\draw[->] (b) -- (c) node [at start]{};
\draw[->] (d) -- (c) node [at start]{};
\end{tikzpicture}

```

```

\subsection{}
First, we need to establish that the strongly connected components of G and GT are the same.

```

By definition, two vertices u and v belong to the same strongly connected component if there exists a path from u to v and a path from v to u .

This property remains unchanged when we transpose the graph since transposing only reverses the direction of edges, but it doesn't affect the existence of paths between vertices.

The component graph represents the strongly connected components of a graph as vertices, with edges indicating connectivity between these components.

Since the strongly connected components remain the same for G and G^T , the component graphs of both G and G^T will have the same set of vertices (representing SCCs).

The edges in the component graph are determined by the edges between SCCs in the original graph.

Transposing a graph doesn't change the connectivity between SCCs, as it only reverses edge directions.

When we transpose G^T , we essentially reverse the direction of edges in G^T , but this doesn't affect the structure of strongly connected components or their connectivity.

Therefore, the component graph of $(G^T)^{SCC}$ remains the same as the component graph of G .

Thus, we've shown that $((G^T)^{SCC})^T = G^{SCC}$, demonstrating that the transpose of the component graph of G^T is indeed the same as the component graph of G .

I think it works, the reasons below:

The first DFS on the transpose of the graph (G^T) identifies the finish times of vertices.

This step remains unchanged in the simplified algorithm.

In the simplified algorithm, the second DFS is performed on the original graph G .

The crucial point is that the order of visiting vertices in the second DFS does not affect the identification of SCCs.

SCCs are determined by the structure of the graph, not the order in which vertices are visited.

Visiting vertices in order of increasing finish times will still correctly identify SCCs because it ensures that we explore vertices within the same SCC before moving to vertices in other SCCs. Since the identification of SCCs is unaffected by the order of visiting vertices in the second DFS, Anya and Albert's simplified algorithm will correctly find SCCs in the graph G .

Therefore, the simplified algorithm proposed by Anya and Albert is valid and will correctly identify the SCCs of the graph G .